

Riepilogo di SQL

STRUTTURA DI UN'INTERROGAZIONE

SELECT <elenco attributi e funzioni>

FROM <elenco delle tabelle o viste>

[WHERE <condizioni>]

[GROUP BY <attributo o attributi di raggruppamento>]

[HAVING <condizione di raggruppamento>]

[ORDER BY <elenco attributi>]

ASC, DESC

NELLA CLAUSOLA WHERE

operatori di confronto

- WHERE attributo op.rel. Valore
- WHERE attributo op.rel. Attributo
- WHERE attributo IS [NOT] NULL
- WHERE attributo IN (val1, val2, ...)
- WHERE attributo BETWEEN val1 AND val2
- WHERE attributo LIKE 'pattern'
- WHERE salario < 30000 (<, =, >, <=, >=, <>)
- WHERE M.salario > E.salario
- WHERE salario IS NOT NULL
- WHERE sesso IN ('M', 'F')
- WHERE salario BETWEEN 300 AND 400
- WHERE nome LIKE '_Mario%'

si può usare anche una subquery annidate

DOVE ANNIDARE LE QUERY

SELECT <elenco attributi e funzioni>	NO
FROM <elenco delle tabelle o viste>	SI
[WHERE <condizioni>]	SI
[GROUP BY <attributo o attributi di raggruppamento>]	NO
[HAVING <condizione di raggruppamento>]	SI
[ORDER BY <elenco attributi>]	NO

SQL e' un linguaggio dichiarativo

RAGIONARE SULLA QUERY

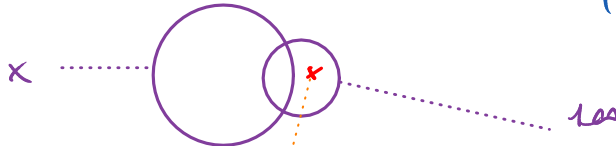
- Le query possono essere di diverse complessità
- In tutti i casi, la richiesta è chiara e ci è molto semplice andare ad identificare ciò di cui abbiamo bisogno all'interno dello schema logico
- Risulta però complicato andare a ragionare su **COME costruire la query** che restituisca effettivamente l'output richiesto
- È SEMPRE importante andare a ragionare bene sulla richiesta, ed eventualmente trovare un altro modo per **riscriverla**, in modo che risulti più semplice andare a trasformarla in una query.

ESEMPIO

- Schema logico
STUDENTE (MATRICOLA, NOME, COGNOME)
ESAME (MATRICOLA, CORSO)

- Query
Gli studenti che hanno sostenuto almeno tutti gli esami sostenuti dallo studente con matricola 100

PROVIAMO A RISCRIVERLA?



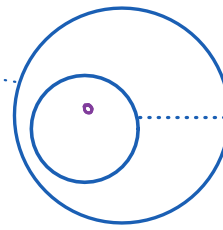
Query

Gli studenti che hanno sostenuto almeno tutti gli esami sostenuti dallo studente con matricola 100

Trovo tutti gli studenti X per cui NON ESISTE un esame che non hanno sostenuto e che lo studente 100 ha sostenuto

L'insieme degli esami sostenuti dallo studente 100 e non sostenuti dallo studente X è vuoto

corsi sost.
dagli stud.
che stiamo
cercando



Corsi sost.
da Matr. 100

FORMULARE
IN SQL
(EVENTUALMENTE ANCHE
IN ALGEBRA REL.)

Tornano utili EXISTS, IN

STUDENTE (MATRICOLA, NOME, COGNOME)
ESAME (MATRICOLA, CORSO)

```
SELECT *  
FROM Studenti AS S  
WHERE NOT EXISTS  
  (SELECT E.Corso  
   FROM Esami AS E  
   WHERE E.Matricola = '100' AND E.Corso NOT IN  
     (SELECT E2.Corso  
      FROM Esami AS E2  
      WHERE E2.Matricola = S.Matricola))
```

Attenzione alla visibilità dei nomi di tabella/attributo.

VINCOLI

DDL (SQL)

- I vincoli analizzati fino ad ora sono supportati dal linguaggio di definizione dei dati in quanto si manifestano nella maggior parte dei casi e nella maggior parte delle applicazioni per DB

- Vincoli sui valori null e valori predefiniti
- Vincoli sulle tuple (CHECK)
- Vincoli sui domini degli attributi
- Vincoli di chiave
- Vincoli di integrità referenziale

PostgreSQL ammette CHECK
solo sul singolo record

- Esistono altri tipi di vincoli i quali, siccome non comuni a tutte le basi di dati, non fanno parte del DDL e devono essere specificati in maniera differente

- Sono chiamati **vincoli di integrità semantica**

Business rules, regole di business,
vincoli di dominio

VINCOLI DI BASE

- **CONSTRAINT** <nome_vincolo> <vincolo>
- <vincolo>:= CHECK <espressione booleana>
 - UNIQUE (<lista_attributi>)
 - PRIMARY KEY (<lista_attributi>)
 - FOREIGN KEY (<lista_attributi_FK>) REFERENCES <nome_tabella> (<lista_att_PK>)
 - [ON DELETE <azione>]
 - [ON UPDATE <azione>]
- <azione>:= NO ACTION | CASCADE | SET DEFAULT | SET NULL

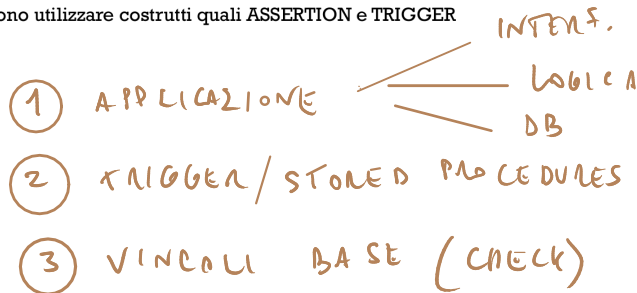
(già visti)

VINCOLI DI INTEGRITÀ SEMANTICA

- «Un impiegato non può guadagnare più del suo supervisore»
- «Ad un progetto non possono lavorare più di n impiegati»
- «Un impiegato non può lavorare più di 48 ore settimanali»

- Questa tipologia di vincoli può essere specificata
 - Sia all'interno dei programmi applicativi
 - Sia tramite un linguaggio di specifica dei vincoli

- Si possono utilizzare costrutti quali ASSERTION e TRIGGER

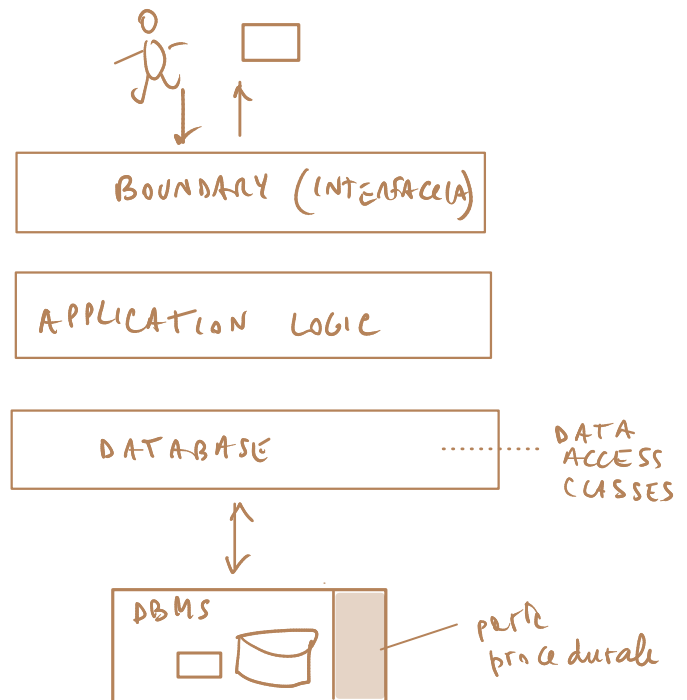


TIPOLOGIA DI VINCOLI

- In linea di massima i vincoli possono essere di 4 tipi
 - Table-level constraints
 - (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK)
 - Column-level constraints
 - (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL)
 - Schema-level constraints
 - Vincoli a livello di schema. Ogni volta che l'intero DB cambia stato, si controlla se un vincolo è violato

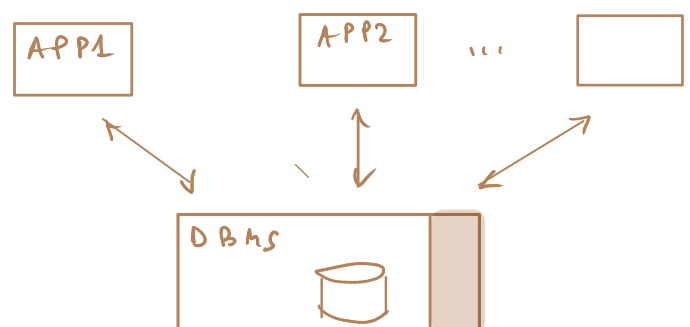
ENTERPRISE DB

La definizione di un vincolo nel DBMS come vincolo di base o vincolo in integrità semantica (implem. come trigger/CHECK/stored procedures) consente di mantenere la consistenza di detto vincolo indipendentemente dalle applicazioni che operano sul DBMS.



RI DONO ANZA

Un vincolo può essere implementato/specificato in più di una parte del sist. informativo.



VINCOLI INTRARELAZIONALI E INTERRELAZIONALI

- La maggior parte dei vincoli sono **intrarelazionali**:
 - Per verificarne l'eventuale rispetto si deve andare a controllare esclusivamente la tabella in cui è avvenuta la modifica
 - PRIMARY KEY** (vado a verificare che non esistano attributi uguali per la chiave primaria nella tabella dove ho effettuato la modifica)
 - UNIQUE** (lo stesso)
 - NOT NULL** (vado a verificare che il valore inserito per un attributo non sia null)
 - CHECK** (Controllo che i valori dati ad uno o più attributi della tabella in cui è definito il vincolo, rispettino un determinato range/dominio)
- Il vincolo di integrità referenziale (FOREIGN KEY) è un vincolo **interrelazionale**, in quanto va a controllare che il valore definito nell'attributo (o negli attributi) esista nella tabella di riferimento.

VINCOLO DI CHECK

- È un costrutto **molto potente**
 - TEORICAMENTE Tutti i vincoli di tabella o di colonna (UNIQUE, PRIMARY KEY, FOREIGN KEY, ...) possono essere espressi tramite un vincolo di CHECK.
 - Almeno in Postgres questo non funziona perché non è possibile creare subquery nei vincoli di check.
 - Far riferimento sia agli attributi della tabella in cui è definito, sia ad attributi di altre tabelle.
 - Anche questo in Postgres non è possibile.

```
create table impiegato(  
  matricola char(8) PRIMARY KEY,  
  dipartimento char(8),  
  check (matricola like 'c%' OR dipartimento in  
    (SELECT nomedip  
     FROM dipartimento));
```

/ intra-rel.

Questo vincolo assicura che ogni impiegato abbia una matricola che cominci per 'c' o appartenga a un dipartimento noto.

```
create table impiegato(  
  matricola char(8) PRIMARY KEY,  
  dipartimento char(8),  
  check (1 >= (SELECT count(*)  
    FROM impiegato I  
    WHERE matricola = I.matricola)  
  );
```

/ intra-rel.

Imposta un vincolo di unicità "manuale" sulla chiave matricola tramite sottoselezione. Controlla che il numero di righe nella stessa tabella impiegato con identica matricola non superi 1:

VINCOLI INTERRELAZIONALI

- Quando un vincolo di CHECK coinvolge due o più tabelle ci possono essere comportamenti indesiderati.
- Nell'esempio sotto, se inseriamo nella tabella **DIPARTIMENTO** più di 100 dipartimenti, il vincolo rimane soddisfatto, dato che il controllo viene fatto **SOLO** quando si inserisce la tupla all'interno della tabella **IMPIEGATO**

Cosa succede se vado a fare l'update di **nomedip** in **DIPARTIMENTO**?

```
create table impiegato(  
  matricola char(8) PRIMARY KEY,  
  dipartimento char(8),  
  check (matricola like 'c%' OR dipartimento in  
    (SELECT nomedip  
     FROM dipartimento));
```

```
create table impiegato(  
  matricola char(8) PRIMARY KEY,  
  dipartimento char(8),  
  check((SELECT count(DISTINCT dipartimento) FROM impiegato) +  
    (SELECT count(nome) FROM dipartimento) < 100));
```



↓
Il vincolo mostrato utilizza due sottoquery per imporre un limite complessivo sul numero di dipartimenti "coinvolti" sia dagli impiegati sia definiti nella tabella dei dipartimenti.

Viene valutato solo in fase di INSERT o UPDATE sulla tabella impiegato, non quando si modificano i dipartimenti.

```
CREATE TABLE impiegato(  
  matricola CHAR(8) PRIMARY KEY,  
  dipartimento CHAR(8),  
  CHECK(  
    (SELECT COUNT(DISTINCT dipartimento) FROM impiegato)  
    + (SELECT COUNT(nome) FROM dipartimento)  
    < 100  
  )  
);
```

- La prima sottoquery conta quanti dipartimenti distinti sono attualmente assegnati agli impiegati.
- La seconda sottoquery conta quanti dipartimenti esistono nella tabella dipartimento.
- Il vincolo impone che la somma di questi due conteggi sia strettamente inferiore a 100.
- Il controllo scatta solo quando si inserisce o aggiorna una riga in impiegato; modifiche dirette alla tabella dipartimento non riattivano il vincolo.
- Se il totale raggiunge o supera 100, qualsiasi nuova tupla in impiegato viene respinta.
- Questo crea un vincolo "interrelazionale", ma con il difetto di valutarsi solo sul lato di impiegato, lasciando potenzialmente incoerenze se si agisce su dipartimento.

In sintesi, il CHECK combina due sottoquery per togliere dai processi di modifica ogni inserimento che farebbe superare la soglia complessiva di 100 dipartimenti coinvolti.

PL/pgSQL

Postgres

PL/SQL (Oracle)

Soluzione dell'esercizio della prima parte.

Top-Down

Tutti gli studenti S tali che non esiste un esame sostenuto da matr. 100 ma non da S

```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS ..... restituisce valore
-- seguono esami sostenuti da booleano (TRUE, FALSE)
-- matr. 100 ma non da S
```

```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS
  (SELECT E.Corso
   FROM Esami AS E
   WHERE E.Matricola = '100' AND E.Corso NOT IN
   -- seguono esami sostenuti da S
```

esami sostenuti
da matr. 100

```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS
  (SELECT E.Corso
   FROM Esami AS E
   WHERE E.Matricola = '100' AND E.Corso NOT IN
   (SELECT E2.Corso
    FROM Esami AS E2
    WHERE E2.Matricola = S.Matricola))
```

esami sostenuti
da matr. 100

esami sostenuti da S